

Credit Name: CSE 3130 Object-Oriented Programming 2
Assignment Name: University.java

The following questions can help you in thinking critically about your problem-solving processes:

Understanding the Problem

How did you approach understanding the challenge?

I first approached the challenge by following the skillbuilder CrestMart we did to draw ideas and have a visual representation of what it should be. This allowed me to have a good place to start and understand how to do the project. Having a small template like that has allowed me to know what I should put in the subclasses so the tester class would work flawlessly.



This is the tester class to make sure its subclasses are correct. I recycled the code there and changed a lot of the variables so it matches the university class.

Were there any parts of the problem you found confusing at first? If so, how did you resolve that confusion?

What confused me at first was understanding if I should give each worker in the university a salary because the crestmart class had associates that didn't have a yearly salary. I resolved this by asking Mr.Abdalla and he was able to tell me what each subclass should have.

```
1 package Skillbuilders;
2 public class associate extends CEmployee
3 {
4     private double hourlyPayRate;
5
6     public associate(String fN, String lN, double rate)
7     {
8         super(fN, lN);
9         hourlyPayRate = rate;
10    }
11
12    public double pay(double hours)
13    {
14        double payEarned;
15
16        if(hours > 40)
17        {
18            payEarned = ((hourlyPayRate*40)
19                + (hourlyPayRate*1.5 * (hours - 40)));
20        }
21        else
22        {
23            payEarned = hours * hourlyPayRate;
24        }
25
26        return payEarned;
27    }
28
29    public String toString()
30    {
31        return(super.toString()+
32            ", Associate"
33            + "Pay rate: "
34            + hourlyPayRate);
35    }
36
37
38
39 }
40
```

This is the associate class and it had an hourly salary but eventually I learned that everyone in the university should have a yearly salary.

Planning the Solution

Did you create a plan or break the problem into smaller steps before coding?

I broke the code into smaller pieces so I would be able to make sure there is no error in the subclasses allowing me to quickly find mistakes if there really is. Breaking the code into smaller steps allowed me to pace myself easier because time gets used up fast if I try to find error in lines of code

```
1 package Mastery; // Places this class inside the Mastery packa
2
3 public class Uemployee {
4     private String FName, Lname; // Stores the employee's first
5     private double salary; // Stores the employee's yearly sal
6
7     public Uemployee(String F, String L, double s) { // Constr
8         FName = F; // Assigns the first name to the instance v
9         Lname = L; // Assigns the last name to the instance va
10        salary = s; // Assigns the salary to the instance vari
11    } // Ends the constructor
12
13    public String getName() { // Method that returns the emplo
14        return FName + " " + Lname; // Combines first and last
15    } // Ends the getName method
16
17    public double getSalary() { // Method that returns the emp
18        return salary; // Returns the stored salary value
19    } // Ends the getSalary method
20
21    public double pay(double weeks) { // Method that calculate
22        return salary / 52 * weeks; // Divides yearly salary i
23    } // Ends the pay method
24 } // Ends the Uemployee class
```

The main structure of the entire code was based on the employee class allowing the whole code to have an exoskeleton to make specific subclasses dedicated to the faculty and staff.

How did you decide on the tools, data structures, or algorithms to use?

I decided on the tools, data structures, and algorithms based on the requirements of the assignment and what we learned in class. Since the program needed to model real-world objects like employees and accounts, I used classes and inheritance to organize the data and reduce repetition. I chose simple data types like strings and doubles because they were enough to store names and balances. For the logic, I used basic control structures like if statements and loops to handle conditions such as minimum balances and user input. Overall, I focused on keeping the design simple, readable, and aligned with object-oriented programming principles.

Implementation

Did you write the code in small pieces or attempt the entire solution at once?

I wrote the code in small pieces because it would make sure I would have no errors and it would allow me to spend less time finding errors. Writing the solution at once is not optimal because I wouldn't know if I am doing it right until the end, causing me to feel demotivated and that would make me not want to continue the project.

How did you test your solution along the way to make sure it was working?

I wasn't able to fully test the solution because there was no main method inside until the tester class. Since I wasn't able to test it I had to hope for the best until the very end. Once I was at the tester class I tested multiple times so I would be able to know if I got anything wrong.

```
(E)mployee \ (P)ay \ (Q)uit
Make a choice: e
Enter Employee number (1 or 2): 1
Sarah Lee Science
```

```
(E)mployee \ (P)ay \ (Q)uit
Make a choice: q
```

Seeing it work allowed me to see how I did and allowed me to catch all of the miniature errors that were in the code.

Overcoming Challenges

What part of the problem was the most difficult for you?

The most difficult part of the program was the do while statement. I have never done do-while statements in a long time and doing it for the first time led to a lot of confusion but with the help of chatGPT and Mr Abdalla I was able to understand it and apply it to my test class.

```
do {
    System.out.println("\n(E)mployee \ (P)ay \ (Q)uit"); //
    System.out.print("Make a choice: ");
    action = input.next();

    if (!action.equalsIgnoreCase("Q")) {

        System.out.print("Enter Employee number (1 or 2): ");
        empNum = input.nextInt();

        switch (empNum) {
            case 1: emp = emp1; break; // Select faculty
            case 2: emp = emp2; break; // Select staff
        }

        if (action.equalsIgnoreCase("E")) {
            if (emp instanceof Ufaculty) {
                System.out.println(emp.getName() + " " + ((Ufa
            }
            else if (emp instanceof Ustaff) {
                System.out.println(emp.getName() + " " + ((Ust
            }
        }
        else if (action.equalsIgnoreCase("P")) {
            System.out.print("Enter number of weeks: ");
            payArg = input.nextDouble();
            payEmployee(emp, payArg); // Call shared pay methc
        }
    }
} while (!action.equalsIgnoreCase("Q")); // Loop until user qu
```

How did you handle moments when you felt stuck or unsure of what to do next?

I would ask Mr Abdalla for help because he would give me insight and instruction that I already covered, allowing me to understand and process the information better. If he is not available I would use chat gpt and it helped me understand what I did wrong and allowed me to fix it.

Learning**Was there anything you learned that you think will help you with future challenges?**

I learned how to make a do while statement that will help me make a code that will continue to run until I manually turn it off. This will help me gain ideas and give me a wider window for creativity for my codes that would need to run forever.